Publishing this: repository, DOI, arXiv

Third edition

Karthikeya
Accretion Labs

11 October 2026

What changed, and why there is a third edition

The first edition had six factual errors. The second edition corrected them and introduced six more operational ones, which a further review caught. Both sets are listed here so you do not act on a remembered version of either.

Fixed in the second edition

1. **The DOI advice was backwards.** Cite the *version-specific* DOI, not the concept DOI. Zenodo says so directly.
2. **The endorsement mechanism was wrong.** Automatic endorsement needs a claimed co-authored paper *and* an institutional email, not "affiliation or prior submission".
3. **The endorser's responsibility was understated.** It is more than archive fit.
4. **"Every journal accepts it" was unsupported**, and CC BY does not "match" the MIT code licence — they govern different artifacts.
5. **"If it builds locally, arXiv will build it too" is not true.** Check arXiv's own log and preview.
6. **The moderation predictions were speculation** and are removed.

Fixed in this third edition

1. **The secret scan was too narrow.** `-only-verified` can miss a key you have just revoked, which is exactly the key you are looking for.
2. **I asserted your arXiv account history** rather than telling you to check it.
3. **Archiving before review was presented as necessary.** It is optional, and if you do it, the archive is a review snapshot and not the replication package.
4. **"The licence should read MIT" and "cross-list `physics.comp-ph` and nothing else"** were written as requirements. They are choices.
5. **Repository facts were asserted, not tested.** There is now a fresh-clone checklist.
6. **A wrong section reference** in the endorser table.

I verified the claims in §3–§6 against the official pages, quoted where it matters. Anything I could not verify is flagged as needing your check — and §7 is the set of things only a fresh clone can settle.

The recommendation, unchanged

Get the independent code review before you post to arXiv.

A posting is permanent. v2 replaces v1 for readers, but v1 stays visible and the abstract page shows the history. Six defects have now been found in this work. Every one was found by reading code or prose against a claim — never by running the pipeline, which was green through all of them. The sixth was in the first edition of this very guide.

§1–§4 can all proceed while the review is outstanding. Only the submission itself waits.

1 Pre-flight

1.1 Rotate the Materials Project key

Do this first. The key was on screen in a screenshot. Generate a new one at https://next-gen.materialsproject.org/api, replace the contents of `C:\research\mp_api_key.txt`, and confirm the old one stops working.

1.2 Scan for the old key without typing it anywhere

The first edition told you to paste the old key into a `git grep`. Don't: that puts the credential into your shell history, and on Windows into `PSReadLine`'s saved history file as well. Use a scanner that looks for credential *shapes* instead:

```
# both scan history, not just the working tree
gitleaks detect --source . --redact
trufflehog git file://.            # NOT --only-verified; see below
```

The second edition told you to pass `-only-verified`. That is wrong for this job: verification works by trying the credential against the provider, so a key you have just rotated fails verification and can be filtered out of the results — and a revoked key still sitting in public history is precisely what you are scanning for. Review the verified, unverified and unknown findings. Check your installed version's flags, since the syntax has changed between releases.

GitHub's own secret scanning also runs automatically on public repositories, which is a reason to check before flipping visibility rather than after. And look in the places a `.gitignore` never covered: cached API outputs, notebook checkpoints, screenshots, and any log you pasted into a document.

A `.gitignore` entry proves nothing about what was committed in the past. If a scanner finds the key in history, rewriting history is straightforward on a repository that has never been public and awkward afterwards — which is the other reason to do this now.

1.3 What will actually be published

- `data/` is ignored, so the two 175 MB GNoME CSVs stay out. `REPRODUCE.md` carries their URLs and SHA-256 sums.
- The four cached Materials Project files are committed deliberately — see §5 for the licence position, which I have now verified.
- `review/` holds the referee report and four response documents. Publishing them is a strength, but read them once as an outsider first.
- About 10 MB without `data/`.

2 Making the repository public

```
gh repo edit --visibility public --accept-visibility-change-consequences
gh repo edit --description "Chemistry-resolved audit of PBE vs r2SCAN stability verdicts in GNoME"
```

```
gh repo edit --add-topic dft --add-topic materials-science --add-topic reproducibility \
             --add-topic r2scan --add-topic gnome --add-topic convex-hull
gh api repos/:owner/:repo --jq '.visibility, .has_issues, .license.spdx_id'
gh run list --limit 3
```

Issues should be on, since `REVIEWER.md` asks for findings there. The licence field should read whatever code licence you have actually chosen — it is `MIT` in the repository today, and confirming that is your decision rather than a publication requirement. Watch the next CI run: going public changes the Actions quota and how secrets behave in forks, so confirm the workflow still *skips* the credentialed phases cleanly rather than failing when `MP_API_KEY` is unavailable.

3 The DOI — cite the version, not the concept

This is the correction that matters most, because getting it wrong makes the replication package ambiguous. Zenodo's own guidance:

> "You should normally always use the DOI for the **specific version** of your record in citations."

A version DOI identifies one exact release. A concept DOI covers all versions and resolves to the latest, which is useful for pointing at an evolving project and wrong for a reproducibility citation — a reader following it in a year gets code that is not the code the paper used.

1. Sign in to https://zenodo.org with GitHub. Zenodo only sees public repositories, so §2 comes first.
2. Enable `FOUNDERACCRETION/gnome-audit` in Zenodo's GitHub settings. It archives releases created *after* the hook is on, so enable before releasing.
3. Cut the release. `.zenodo.json` is in the repository, so the metadata comes from there rather than a web form:

   ```
   gh release create v1.0.0 \
     --title "v1.0.0 - accompanies the preprint" \
     --notes "Analysis code, pre-registered decision rules and derived results.
   53 headline checks and 242 table-value checks green. See REPRODUCE.md and REVIEWER.md."
   ```

4. Zenodo mints both DOIs. **Put the version DOI and the release tag in the paper**; the concept DOI is fine for a README badge.
5. Edit three places: the paper's Data and code availability section (it currently says the deposit has not been made), `CITATION.cff`'s `doi:` field, and the README badge. Then rebuild the PDF and re-run `python run_all.py`.

Archiving before the review is optional. A reviewer needs a fixed thing to look at, and a tagged commit does that job — `git tag review-2026-10-11 && git push -tags` is enough, and costs nothing if the review changes the code. Two workable routes:

- *Tag only.* Send the reviewer a tag, archive once after their findings are addressed, and cite that version DOI. Simplest, and the DOI matches the submitted paper by construction.
- *Archive now as a snapshot.* If you want a citable object immediately, archive it and **label it a review snapshot in the release notes** — not the replication package. Then archive again after the review and cite the later version DOI.

Either way the rule is the same: the version DOI in the paper must correspond to the code and outputs that produced the paper's numbers. Do not cite a snapshot as the replication package for a paper whose numbers came from later code.

4 arXiv

4.1 Endorsement: the mechanism, corrected

arXiv's automatic route requires *both* of:

- that you have "claimed ownership of a paper submitted by a co-author", and
- that your email address meets arXiv's institutional email criteria.

arXiv says you "may receive" automatic endorsement when both hold, and does not publish the email criteria. The second edition stated flatly that you have no co-authored arXiv paper; I have not checked your account, so treat it as a conditional: *if* you have never claimed authorship of a co-authored arXiv submission, the first condition fails and the second is moot. Either way, **do not act on a prediction**. Create the account, start a submission, and follow the instructions arXiv gives *your* account. That is faster and more reliable than reasoning about the policy.

4.2 What you are actually asking an endorser for

The first edition called it "archive-fit endorsement". That understates it. arXiv's wording:

> "We ask that you only endorse individuals when you are confident they will submit high-quality research papers."

Endorsers should personally know the author or have seen the paper. They should decline if the author "is unfamiliar with the basic facts of the field" or if "the work is entirely disconnected with current work in the area". They are not expected to read the paper in detail or verify its correctness — but it is a judgement about you, not a formality.

Mechanically: start a submission, arXiv issues a six-character endorsement code, you send the code and the paper, the endorser enters it with a yes or no. The "Which of these authors are endorsers?" link on any abstract page shows who is eligible. **Ask one person at a time**; arXiv explicitly says not to email several at once or the same person twice.

A request that matches what arXiv is actually asking:

> Would you consider endorsing me for a `cond-mat.mtrl-sci` submission? I understand endorsement is not peer review, so I am sending the manuscript and the replication package so you can judge whether it is suitable work for the archive. [One sentence on what the paper does and why it touches their work.] My endorsement code is XXXXXX. I am an independent researcher at Accretion Labs.

Two notes on that. Do not describe yourself as "unaffiliated" while listing Accretion Labs in the author block — pick one description and make it accurate. And the people below are **candidates I have not verified**: before approaching anyone, check their current affiliation and confirm eligibility through the endorser link on one of their arXiv abstract pages.

Candidate	Base (verify)	Why they are a sensible ask
G. Pizzi	PSI / EPFL	Reents–Bercx–Pizzi is the closest prior framing, as your related-work section on flip-rate framing says
G. Csányi	Cambridge	Warford–Thiemann–Csányi's selective-U argument is what Section 4.3 tests
G.-M. Rignanese	UCLouvain	Yu et al.'s per-element error map is your methodological precedent
C. J. Bartel	Minnesota	decomposition-reaction framing is upstream of your outcome definition
Z. Wang	UC San Diego	An et al. ran the SCAN re-screen your Section 2 compares against

4.3 What to upload

Source, not PDF — and this is a requirement, not a preference: arXiv does not accept a PDF generated from TeX source in place of that source.

```
cd paper
tar czf arxiv_submission.tar.gz gnome_audit_paper.tex \
    fig1_flip_by_chemistry.png fig2_budget.png fig3_gnome_vs_known.png \
    fig4_per_element.png fig5_bandgap.png fig6_logit.png
```

- No `.bib` is needed: the references are an inline `thebibliography`, so nothing has to run BibTeX.
- `\graphicspath{{./}{../results/}}` resolves via the `./` branch inside the tarball; the second branch is harmless.
- Extract the tarball into an empty directory and compile *that*, not your working copy, so a file you forgot to include shows up now.
- Then — and this is the part the first edition got wrong — **read arXiv's own compilation log and look at its PDF preview before submitting**. A local build is evidence, not a guarantee: compiler version, package availability, filename handling and configuration all differ. Nor does source submission guarantee a usable HTML rendering; treat that as a bonus if it appears.

4.4 Metadata

Primary `cond-mat.mtrl-sci`.

Cross-list `physics.comp-ph` is the natural second home given how much of the paper is method and reproducibility, but it is a judgement about your paper's scope rather than a rule. Pick deliberately and keep the list short; scattering cross-lists does not help.

Abstract Plain text with the light TeX arXiv accepts (`$^2$`, `$U$`, `$d$`). Check the preview; a broken abstract is the most visible possible typo.

Comments *49 pages, 6 figures. Code, pre-registered decision rules and derived results:* `github.com/FOUNDERACCRETION/gnome-audit` *archived at doi:… (v1.x). Includes an independent-review packet.*

Affiliation "Accretion Labs".

ORCID Get one at https://orcid.org and link it. Free, and required for the ChemRxiv route anyway.

4.5 Licence

The licence on a posted version is irrevocable. arXiv offers its own non-exclusive distribution licence and a set of Creative Commons options including CC BY 4.0 and CC BY-SA 4.0. Different *versions* of a paper can carry different licences, but you cannot change the one already posted.

Two corrections to the first edition. I claimed the non-exclusive licence is accepted by "every journal": I have no basis for that and arXiv itself tells you to check journal and funder policies. And CC BY does not "match" the MIT licence on the code — the manuscript and the code are separate artifacts under separate licences, and choosing one says nothing about the other. The third artifact, the Materials Project extracts, is governed by neither (see §5).

So: decide the manuscript licence after checking the policy of whatever journal you might submit to later. If you have no specific journal in mind, CC BY 4.0 is the choice consistent with a paper whose point is that others can check it. If you do, read that journal's policy first — this is the one decision here you cannot walk back.

4.6 After submitting

Submissions go through moderation for topical and scholarly suitability, and endorsement does not remove that step: arXiv can still reject or reclassify. A reclassification suggestion is routine and not a verdict on the work.

The first edition also predicted that a sole author with a company affiliation would get closer scrutiny, and that the preregistration and AI disclosure were "exactly what a moderator wants to see". I have no evidence for either and both are now removed. Submit the work and let the process run.

5 The Materials Project extracts — verified

The first edition asserted the licence position. I have now checked it. The Materials Project terms of use place downloaded content under CC BY 4.0 and state that it

> "may be copied, distributed, transmitted, and adapted, without obtaining specific permission, provided proper attribution is given to the Materials Project"

The same terms permit collecting large fractions of the database through the API and presenting processed results with attribution, and prohibit scraping the website directly. The four committed files came through the API, so redistributing them with attribution is within the terms. `results/PROVENANCE.md` now quotes this, names the required attribution, and states explicitly that **the repository's MIT licence cannot grant rights to third-party data** — the MP extracts are governed by the MP terms and the GNoME CSVs, which are not redistributed here, by whatever DeepMind applies.

Still worth your own check before publishing: that no credential or private value sits inside those cached CSVs (they should contain only `material_id`, `energy_above_hull`, `nelements` and `chemsys`), and that the README distinguishes cached API pulls from regenerated results. It does.

6 ChemRxiv, the fallback

Verified: an ORCID iD is required to register and upload, submissions are PDF with supplementary material in any format, and the licence choices offered are CC BY 4.0, CC BY-NC 4.0 and CC BY-NC-ND 4.0. Content must be original research, in English, not already accepted for publication, and it passes a plagiarism check. ChemRxiv's author FAQ puts screening at one to two business days and posting one to two days after that.

Two corrections. ORCID is the *registration* gate, not the only gate — ChemRxiv curates and screens submissions, and approval does not imply endorsement by its publisher or partner societies. And treat the timings and licence list as current-at-time-of-writing details to confirm against its submission guide rather than as guarantees.

It is a real venue, not a consolation prize. The honest trade-off is audience: your paper lands under computational chemistry rather than among the `cond-mat.mtrl-sci` readers who would actually use the budget rule. Try arXiv first; use ChemRxiv if the endorsement route stalls for two or three weeks.

7 Order of operations

1. Rotate the Materials Project key. (§1.1, today)
2. Scan history with a secret scanner — never by typing the key. (§1.2)
3. Read `review/` once as an outsider. (§1.3)
4. Tag a review commit and **send it with `REVIEWER.md` to an independent reviewer.** (the long pole — start now)
5. Make the repository public; check Actions, issues, licence. (§2)
6. Verify the four fresh-clone facts. (§7)
7. Optionally enable Zenodo and archive a labelled review snapshot. (§3)
8. Get an ORCID; create an arXiv account; start a submission to see what your account is actually told about endorsement. (§4.1)
9. Ask one endorser. Wait. Ask the next if needed. (§4.2)
10. **Wait for the review. Fix what it finds.** (the hold point)
11. If anything scientific changed, cut and archive a new version; put *that* version DOI in the paper; rebuild; re-run `run_all.py`. (§3)
12. Decide the manuscript licence against a real journal policy. (§4.5)
13. Upload the source tarball; read arXiv's compilation log and preview; submit. (§4.3)
14. Add the arXiv ID to `README.md` and `CITATION.cff`.

Steps 1–8 run in parallel with step 4. Step 9 blocks steps 10–12, and step 10 exists because of step 9.

8 Verify these from a fresh clone, not from this guide

Everything this guide says about the repository is a reported project fact. I have looked at the working tree on your machine, which is not the same as what a clone gets — a `.gitignore` change, for instance, does nothing about what is already tracked or untracked until you commit. Before publishing, run this from a clone in an empty directory:

```
git clone https://github.com/FOUNDERACCRETION/gnome-audit fresh && cd fresh
# 1. are the four cached Materials Project files actually tracked?
git ls-files results/ | grep -E 'mp_gga_raw|mp_r2scan_raw|known_baseline|mp_completeness'
# 2. is anything that should be ignored present, or anything needed missing?
ls results/ | wc -l ; cat .gitignore
# 3. does a no-credential run work, and does it label stored values as stored?
#    (download the two CSVs into data/ first, per REPRODUCE.md section 2)
find results -type f \( -name '*.csv' -o -name '*.png' -o -name '*.json' \) \
     ! -name 'known_baseline.csv' ! -name 'mp_completeness.csv' \
     ! -name 'mp_gga_raw.csv' ! -name 'mp_r2scan_raw.csv' -delete
unset MP_API_KEY MP_API_KEY_FILE
python run_all.py 2>&1 | tee /tmp/fresh.log
```

```
grep -c 'stored, not regenerated here' /tmp/fresh.log   # must be non-zero
grep -i 'api.key\|credential' /tmp/fresh.log            # must find no demand for one
```

Four things that must hold, and none of which this guide can establish for you:

- the four cached inputs are tracked, so a clone has them;
- the wipe in step 3 preserves exactly those four and nothing else;
- the advertised reproducible path needs no credential, and says which values are stored rather than regenerated;
- `results/PROVENANCE.md` and the licence files are present in the clone.

Then confirm the release metadata matches the final version: the title, the version DOI, and the tag the paper cites.

9 What the repository carries for a reviewer

- `REVIEWER.md` — reproduction versus correctness; all defects found so far and how each was found, as calibration for how much the green checks are worth; eight ordered areas to inspect with the specific thing to check in each; and a request to independently reimplement the headline cell rates and the budget comparison rather than rerun ours.
- `REPRODUCE.md` — built around the step that makes the rest meaningful (delete the stored results first), with input checksums and a troubleshooting order.
- `results/runtimes.csv` — per-phase wall time, total, platform, Python version, written on every run.
- `results/PROVENANCE.md` — what is regenerated, what is cached, the verified Materials Project terms, and the limits of the MIT licence.
- `CITATION.cff`, `.zenodo.json`.
- `NB` and `SEED` environment variables on the bootstrap phases, so a reviewer can rerun a stability check cheaply or expensively without editing code.